DALHOUSIE UNIVERSITY

# Cleaning and Transforming Data

## Feature engineering, data reduction, category coding, and monitoring records

**Prof. Gabriel Spadon, PhD**

Faculty of Computer Science

spadon@dal.ca

# 1 Feature engineering and units

**What it is for** A derived variable combines source measurements into a quantity that answers the question better than either source alone.

**The rule** Write the formula with its units and convert every input to those units before computing. Body mass index (BMI) is

$$\text{BMI} = \frac{\text{mass in kg}}{(\text{height in m})^2}.$$

**Worked example** For a mass of 82 kg and a height of 178 cm, convert the height to 1.78 m first. Then

$$\text{BMI} = \frac{82}{1.78^2} = 25.88 \text{ kg/m}^2.$$

The code applies the same formula to three people, giving 25.88, 22.41 and 29.00. BMI here illustrates arithmetic and documentation, not a clinical judgement.

**R**

```r
measurements <- data.frame(
  mass_kg = c(82, 61, 95),
  height_cm = c(178, 165, 181))
measurements$bmi <- measurements$mass_kg /
  (measurements$height_cm / 100)^2
round(measurements$bmi, 2)
# 25.88 22.41 29.00
```

**Python**

```python
import pandas as pd

measurements = pd.DataFrame({
    "mass_kg": [82, 61, 95],
    "height_cm": [178, 165, 181]})
measurements["bmi"] = (
    measurements["mass_kg"]
    / (measurements["height_cm"] / 100)**2)
print(measurements["bmi"].round(2).to_list())
# [25.88, 22.41, 29.0]
```

Both versions keep the original measurements beside the derived value. The columns come from one table, so corresponding rows describe the same person. R combines vectors by position and can recycle a shorter vector. pandas combines Series by their index labels. When the inputs come from different tables, establish the row relationship first. Equal lengths do not prove that rows correspond.

# 2 Reducing a dataset and sampling

**What it is for** Reduction makes a dataset smaller along one of three axes. Each discards something, so choose it for an analytical reason, not because the file gets smaller.

**The three choices** Figure 1 separates them.

- **Fewer variables.** Keep only the variables the question needs. Fewer variables simplify computation and display.
- **Fewer observations.** Makes a first inspection manageable, but may omit rare conditions.
- **Fewer distinct values.** Binning, taught in Class 03, turns numbers into categories and loses the detail inside each bin.

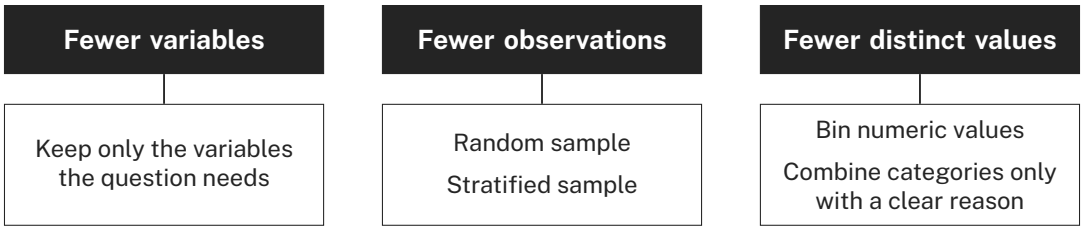



**Figure 1.** Three separate reduction choices.

**Random and stratified samples** A random sample chooses records through a defined chance mechanism. A stratified sample first splits the records into groups, then samples within each group. Stratification helps keep rare groups, and the grouping variable and allocation must follow the question. It cannot repair a biased source dataset.

**Worked example** Ten records come from two sites, five from each. Both samples below contain six records. The random sample may take any mix of sites. The stratified sample takes three from each site, so its site proportions (50 percent each) match the source table.

**R**

```r
data <- data.frame(row_id = 1:10,
  site = rep(c("A", "B"), each = 5))
set.seed(3141)
idx <- sample(seq_len(nrow(data)), size = 6)
random_sample <- data[idx, ]
# Draw three rows from each site.
idx_by_site <- split(
  seq_len(nrow(data)), data$site)
chosen <- unlist(
  lapply(idx_by_site, sample, size = 3))
stratified_sample <- data[chosen, ]
table(stratified_sample$site)
# A 3, B 3
```

**Python**

```python
import pandas as pd

data = pd.DataFrame({
    "row_id": range(1, 11),
    "site": ["A"]*5 + ["B"]*5})
random_sample = data.sample(
    n=6, random_state=3141)
# Draw three rows from each site.
stratified_sample = data.groupby("site").sample(
    n=3, random_state=3141)
print(stratified_sample["site"].value_counts())
# A 3, B 3
```

**Differences to know**

- A seed makes a draw reproducible within one software and algorithm. `set.seed(3141)` and `random_state=3141` do not give the same rows.
- Pass an integer sample size. Seventy percent of 101 rows gives 70 rows after R's integer truncation, while pandas `sample(frac=0.7)` rounds to 71. Sampling a fraction within each group can change the total again.

With proportional allocation, every record has the same chance of selection. If groups are sampled at different rates, a population estimate must weight by those probabilities. Records from the same station or period may be dependent, so randomly chosen rows are not automatically independent observations. Lab 02 does not require subsampling.

## 3 Representing categories

**What it is for** Some software needs numbers, not labels. The representation must not add information the labels do not have.

**The rule** Encode a nominal variable with indicator columns. A row has 1 in the column of its category and 0 elsewhere, so $C$ indicators record membership in $C$ categories. Coding sites A, B and C as 1, 2

and 3 would claim an order and equal spacing that the labels do not have. A regression keeps only $C - 1$ of the indicators and drops one level, which is implied when all the others are 0.

**Worked example** The site labels A, B, A, C give three indicator columns. The first and third rows have 1 under A, the second under B and the fourth under C.

**R**

```r
site <- factor(c("A", "B", "A", "C"))
indicators <- model.matrix(~ site - 1)
as.data.frame(indicators)
# Three columns, one per site.
```

**Python**

```python
import pandas as pd

site = pd.Series(["A", "B", "A", "C"])
indicators = pd.get_dummies(site, dtype=int)
print(indicators)
# Three columns, one per site.
```

Neither version assigns an order to the site names, and both keep all three columns. Lab 02 needs clear variable types and documented labels, not indicator encoding.

## 4 From monitoring records to a documented table

**What it is for** This section works through one monitoring record, the kind of file Lab 02 uses. The goal is an analytical table in which every change can be traced back to the raw record.

### 4.1 Separate the meaning of unavailable measurements

**The rule** Read every field as text first, list the distinct tokens, and give each one a status before any numeric conversion. A value that cannot be used in exact arithmetic may still carry information.

**Worked example** A concentration column is documented in mg/L. Table 1 shows five strings with four different statuses.

**Table 1.** A small monitoring example with distinct value-status categories.

| Raw field | Status | Interpretation |
|---|---|---|
| `0` | measured | Exact recorded zero in the stated unit. |
| blank | missing | No usable measurement was recorded. |
| `<0.1` | censored | Below 0.1 mg/L; retain 0.1 as a limit, not a value. |
| `oops` | invalid | Unexpected token; investigate the original record. |
| `0.24` | measured | Exact recorded value, subject to provider quality flags. |

- `0` and `0.24` are exact measurements. A measured zero is a value.
- `<0.1` is censored. It reports an upper limit, not a measurement of 0.1 and not a zero.
- The blank is missing under this example's documentation.
- `oops` is invalid under the numeric schema and needs investigation.

Provider flags may add restrictions, such as a provisional reading or a failed instrument check. The code below assigns the statuses by hand, keeps the raw strings and units, and checks that the raw text is unchanged.

**R**

```r
raw <- data.frame(
  value_text = c("0", "", "<0.1", "oops",
                 "0.24"),
  unit = rep("mg/L", 5),
  stringsAsFactors = FALSE)
cleaned <- raw
cleaned$status <- c(
  "measured", "missing", "censored",
  "invalid", "measured")
cleaned$value_mg_L <- c(0, NA, NA, NA, 0.24)
cleaned$limit_mg_L <- c(NA, NA, 0.1, NA, NA)
stopifnot(identical(
  raw$value_text, cleaned$value_text))
# These statuses follow the stated rules.
# Original strings and units remain.
```

**Python**

```python
import pandas as pd

raw = pd.DataFrame({
    "value_text":
        ["0", "", "<0.1", "oops", "0.24"],
    "unit": ["mg/L"]*5})
cleaned = raw.copy()
cleaned["status"] = [
    "measured", "missing", "censored",
    "invalid", "measured"]
cleaned["value_mg_L"] = [
    0, None, None, None, 0.24]
cleaned["limit_mg_L"] = [
    None, None, 0.1, None, None]
assert raw["value_text"].equals(
    cleaned["value_text"])
```

Hand assignment works for five known records. A real file needs a rule taken from the provider's data dictionary. Keep literal tokens such as `NA` as text until the documentation says whether they mean unavailable, a category, or something else. Do not suppress conversion warnings without looking at the records that caused them.

## 4.2 Apply the documented rule to every row

**The rule** For this example only, a blank means missing, an unsigned decimal string is an exact measurement, and a less-than sign followed by such a decimal is a reporting limit. Anything else is invalid and is not silently turned into missing. The regular expressions encode these rules.

- `[0-9]+` matches one or more digits.
- `([.][0-9]+)?` allows an optional decimal part.
- The anchors `^` and `$` in R, or the full-match operation in Python, require the whole string to match.

**Worked example** Applied to the five strings, the rule reproduces the statuses of Table 1. Exact values go to `value_mg_L`, the limit 0.1 goes to a separate `limit_mg_L` column, and the exact values are converted with $1\ \mathrm{mg/L} = 1000\ \mu\mathrm{g/L}$, so 0.24 mg/L becomes 240 µg/L.

**R**

```r
raw <- data.frame(
  value_text = c("0", "", "<0.1", "oops",
                 "0.24"),
  unit = rep("mg/L", 5))
cleaned <- raw
text <- trimws(raw$value_text)
# This provider defines blank as missing.
missing <- text == ""
censored <- grepl("^<[0-9]+([.][0-9]+)?$", text)
measured <- grepl("^[0-9]+([.][0-9]+)?$", text)
cleaned$status <- ifelse(missing, "missing",
  ifelse(censored, "censored",
    ifelse(measured, "measured", "invalid")))
cleaned$value_mg_L <- NA_real_
cleaned$value_mg_L[measured] <-
  as.numeric(text[measured])
cleaned$limit_mg_L <- NA_real_
cleaned$limit_mg_L[censored] <-
  as.numeric(sub("^<", "", text[censored]))
cleaned$value_ug_L <- 1000 * cleaned$value_mg_L
cleaned
```

**Python**

```python
import pandas as pd
raw = pd.DataFrame({
    "value_text":
        ["0", "", "<0.1", "oops", "0.24"],
    "unit": ["mg/L"]*5})
cleaned = raw.copy()
text = raw["value_text"].str.strip()
# This provider defines blank as missing.
missing = text.eq("")
censored = text.str.fullmatch(
    r"<[0-9]+([.][0-9]+)?")
measured = text.str.fullmatch(
    r"[0-9]+([.][0-9]+)?")
cleaned["status"] = "invalid"
for mask, label in [(missing, "missing"),
                    (censored, "censored"),
                    (measured, "measured")]:
    cleaned.loc[mask, "status"] = label
cleaned["value_mg_L"] = pd.to_numeric(
    text.where(measured))
cleaned["limit_mg_L"] = pd.to_numeric(
    text.where(censored).str.removeprefix("<"))
cleaned["value_ug_L"] = (
    1000 * cleaned["value_mg_L"])
print(cleaned)
```

These rules are deliberately narrow. Real records may use signs, scientific notation, other missing codes, or other limit formats, so inspect the tokens before adapting them. The documentation and the token audit decide the classification, and the code makes it reproducible across all rows. The cleaned copy never replaces a censored result with zero or half the limit. That would need an extra assumption and a sensitivity analysis, and Helsel shows how such substitution distorts later estimates [1].

## 4.3 Check units, keys and counts

**The rule** Keep the downloaded file unchanged. Before combining observations, identify the station, time, parameter, unit and quality-flag fields, and state the observational unit. Many tables hold one row per station, time and parameter, not one row per station.

**Duplicates** The observational unit defines a duplicate. Choose the key columns, for example station, date and parameter, and count rows that share a key before removing anything.

- R. `duplicated(d[key]) | duplicated(d[key], fromLast = TRUE)` marks every row in a repeated group.
- Python. `d.duplicated(subset=key, keep=False)` does the same.

**Worked example** Four nitrate rows come from two stations. Station S1 has two rows on 2026-09-01, so the key station, date and parameter repeats. Both rows are marked, and neither is removed. The two values, 0.24 and 0.31 mg/L, differ, so the repeat is not an exact double export.

**R**

```r
d <- data.frame(
  station = c("S1", "S1", "S1", "S2"),
  date = c("2026-09-01", "2026-09-01",
           "2026-09-02", "2026-09-01"),
  parameter = rep("nitrate", 4),
  value_mg_L = c(0.24, 0.31, 0.22, 0.18)
)
key <- c("station", "date", "parameter")
repeated <- duplicated(d[key]) |
  duplicated(d[key], fromLast = TRUE)
sum(repeated)   # 2
d[repeated, ]
#   station       date parameter value_mg_L
# 1      S1 2026-09-01   nitrate       0.24
# 2      S1 2026-09-01   nitrate       0.31
```

**Python**

```python
import pandas as pd

d = pd.DataFrame({
    "station": ["S1", "S1", "S1", "S2"],
    "date": ["2026-09-01", "2026-09-01",
             "2026-09-02", "2026-09-01"],
    "parameter": ["nitrate"] * 4,
    "value_mg_L": [0.24, 0.31, 0.22, 0.18],
})
key = ["station", "date", "parameter"]
repeated = d.duplicated(subset=key, keep=False)
print(repeated.sum())   # 2
print(d[repeated])
#   station        date parameter  value_mg_L
# 0      S1  2026-09-01   nitrate        0.24
# 1      S1  2026-09-01   nitrate        0.31
```

A repeated key is a question for the documentation. Two rows may be an accidental double export, or they may differ in depth, sample fraction or laboratory replicate, which means the key was incomplete.

**Units** Convert only with a named rule, such as $1\,\mathrm{mg/L} = 1000\,\mu\mathrm{g/L}$. Keep the original unit field and put the converted value in a new, clearly named column. Do not combine concentrations of different parameters because they share a unit. Flagging an unusual reading is a reason to investigate, not evidence that it should be deleted.

**Counts** Report the number of exact, missing, censored, invalid and excluded records. A mean of the exact measurements describes that subset. When censoring or exclusions are substantial it may not represent all sampled conditions, so state the denominator and the limitation beside the summary.

## 4.4 The Lab 02 workflow

Table 2 and Figure 2 give the order of work for one monitoring record.

**Table 2.** The Lab 02 order of work.

1. Read one approved monitoring file and identify its observational unit.
2. Audit missing values, reporting limits, types, units and quality flags.
3. Build a cleaned copy with a reason for each changed or excluded value.
4. Compare summaries before and after cleaning, keeping the raw file unchanged.
5. If categories help the question, compare two binning rules on one suitable numeric variable.

Lab 02 requires no model fitting. Use one approved language. Follow the syllabus for the private submission and later portfolio release, and preserve the same submitted commit and its recorded history.

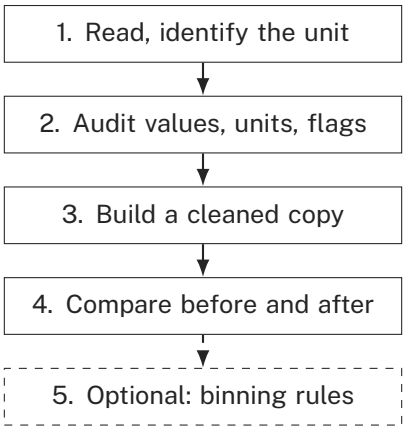



**Figure 2.** The five Lab 02 steps; the dashed step is optional.

## 5 Summary

- A derived variable is only as correct as its units. Convert before you compute.
- Every reduction discards something. Stratified sampling keeps each group represented.
- Indicators record membership without inventing an order. A regression keeps $C - 1$ of them.
- In a monitoring record, measured, missing, censored and invalid are different statuses. Keep the raw string, store limits apart from values, and report every count.
- Count repeated keys before removing any row.

## References

[1] D. R. Helsel. “Fabricating data: how substituting values for nondetects can ruin results, and what can be done about it”. In: *Chemosphere* 65.11 (2006), pp. 2434–2439.